Studio Nx

Backend(FastAPI/Python) 66 — get_recent_projects()

2026-10-12

Backend(FastAPI/Python) 66 — `get_recent_projects()` (최근 프로젝트 · 작업 시간)

항목	내용
파일	`studionx/backend/app/services/dashboard_service.py`
작업	P1-16 대시보드 API: 마이페이지의 최근 프로젝트 5개와 최근 7일 작업 세션 (PRD 01 FR-121, FR-128)
작성	Claude (AI), 2026-10-12, 사용자 검토
이전 · 다음	65 `get_ai_usage()` · 67 `get_ready_projects()`

1. 하는 일

이 함수는 내가 참여한 프로젝트 중 **최근에 수정한 5개**와, 각 프로젝트에서 **최근 7일 동안 내가 시퀀서를 연 기록(세션)**을 돌려줍니다. 마이페이지(P1-17)는 이 데이터로 "최근 프로젝트" 목록과 요일별 작업 시간 막대를 그립니다.

데이터	결과
내 프로젝트 7개, 삭제한 프로젝트 1개, 남의 프로젝트 1개, 멤버로 참여한 남의 프로젝트 1개	내 것과 멤버로 참여한 것 중 삭제되지 않은 프로젝트를 최근에 수정한 순서로 **5개** 돌려줍니다
프로젝트 A에 내 세션이 1일 전, 3일 전, 8일 전에 있고 다른 사람의 세션도 있음	A의 `sessions`에는 **3일 전 → 1일 전** 세션 두 개만 들어갑니다 (8일 전 세션과 다른 사람의 세션은 빠집니다)
프로젝트가 없음	`[]`를 돌려줍니다

응답 한 줄의 예:

```
{"project_id": 12, "title": "Night Drive", "bpm": "128.50", "music_key": "A minor",
 "updated_at": "2026-10-12T02:10:11.123000",
 "sessions": [{"started_at": "2026-10-09T03:00:00", "last_seen_at": "2026-10-09T03:30:00", "active_seconds": 300}]}
```

2. 언제 쓰나

호출하는 곳	화면
68 `get_dashboard`(예정) → `GET /me/dashboard`	myPage.html의 Recent projects와 이번 주 작업 시간 (P1-17)

요일은 화면이 나눕니다. 서버는 세션 시각을 **UTC 그대로** 보내고, 화면이 브라우저 시간대에 맞춰 "월, 화, 수 …"로 나눠 더합니다 (DB.md §3.1). 그래서 서버는 사용자의 시간대를 몰라도 됩니다. 여기서 "7일"은 지금으로부터 7 × 24시간을 뜻합니다 (`started_at >= 지금 - 7일`).

Phase 1에는 세션을 만드는 기능(시퀀서, Phase 2)이 없어서 `sessions`는 비어 있습니다. 시험에서는 세션 행을 직접 넣어서 확인했습니다.

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import select
from sqlalchemy.orm import Session

from app.models import Project, ProjectMember, ProjectSession, User
from app.schemas.dashboard import (
    DashboardAi,
    DashboardResponse,
    DashboardStorage,
    ReadyProject,
    RecentProject,
    SessionItem,
    StorageByKind,
    UsageCount,
)
```

`datetime` 줄과 `select`, `Project`, `ProjectMember`, `ProjectSession`, `SessionItem`을 이번에 추가했습니다. 이 함수는 상수 `RECENT_PROJECT_COUNT`(5)와 `RECENT_DAYS`(7)도 사용합니다.

함수

```
def get_recent_projects(db: Session, user: User) -> list[RecentProject]:
    projects = db.scalars(
        select(Project)
        .join(ProjectMember, ProjectMember.project_id == Project.project_id)
        .where(ProjectMember.user_id == user.user_id, Project.deleted_at.is_(None))
        .order_by(Project.updated_at.desc(), Project.project_id.desc())
        .limit(RECENT_PROJECT_COUNT)
    ).all()
    if not projects:
        return []

    since = datetime.now(UTC).replace(tzinfo=None) - timedelta(days=RECENT_DAYS)
    rows = db.scalars(
        select(ProjectSession)
        .where(
            ProjectSession.project_id.in_([p.project_id for p in projects]),
            ProjectSession.user_id == user.user_id,
            ProjectSession.started_at >= since,
        )
        .order_by(ProjectSession.started_at)
    ).all()
    sessions = {p.project_id: [] for p in projects}
    for row in rows:
        sessions[row.project_id].append(
            SessionItem(
                started_at=row.started_at,
                last_seen_at=row.last_seen_at,
                active_seconds=row.active_seconds,
            )
```

```
        )

    return [
        RecentProject(
            project_id=p.project_id,
            title=p.title,
            bpm=p.bpm,
            music_key=p.music_key,
            updated_at=p.updated_at,
            sessions=sessions[p.project_id],
        )
        for p in projects
    ]
```

문법 풀이

import 문법	뜻
`from datetime import UTC, datetime, timedelta`	지금 시각(UTC)과 기간("7일")을 다룹니다
`from sqlalchemy import select`	SELECT 문을 만듭니다
`from app.schemas.dashboard import SessionItem`	세션 한 개의 응답 모양입니다 (시작 시각, 마지막 신호, 작업 시간 초)

식	뜻	예
`db.scalars(select(Project)…).all()`	행마다 `Project` 하나씩 담긴 목록을 가져옵니다	
`.join(ProjectMember, …).where(ProjectMember.user_id == 나)`	내가 멤버인 프로젝트만 고릅니다 (OWNER가 아니어도 됩니다)	
`.limit(RECENT_PROJECT_COUNT)`	5개만 가져옵니다	
`datetime.now(UTC).replace(tzinfo=None) - timedelta(days=7)`	지금(UTC)으로부터 7일 전 시각입니다. DB와 같은 "시간대 정보가 없는 UTC" 형식입니다	
`.in_([p.project_id for p in projects])`	`project_id IN (12, 9, 7, …)`: 다섯 프로젝트를 한 번에 조회합니다	
`{p.project_id: [] for p in projects}`	프로젝트 번호마다 빈 목록을 만듭니다. 그래서 세션이 없는 프로젝트도 `[]`가 됩니다	`{12: [], 9: []}`
`sessions[row.project_id].append(…)`	해당 프로젝트의 목록에 세션을 추가합니다	
`[RecentProject(…) for p in projects]`	프로젝트 순서(최근 수정순)를 그대로 유지해서 응답 목록을 만듭니다	

4. 흐름

1. 내가 멤버이고 삭제되지 않은 프로젝트를 최근에 수정한 순서로 5개 가져옵니다. 하나도 없으면 `[]`를 돌려줍니다.
2. 그 5개 프로젝트에 있는 **내** 세션 중 최근 7일 안의 것을 **한 번에** 시작 시각 순서로 가져옵니다.
3. 세션을 프로젝트별로 나눠 담고, 프로젝트 순서대로 응답을 만듭니다.

5. 왜 이렇게

선택	이유
세션을 쿼리 **한 번**으로 조회	프로젝트마다 따로 물으면 5번이 나갑니다 (N+1 문제). `IN (...)`으로 한 번에 조회했고, 시험에서 SELECT가 **2번**(프로젝트, 세션)만 나가는 것을 확인했습니다 (템플릿 권장 방식)
내 세션만 셈	마이페이지에 보여 줄 것은 "내 작업 시간"이기 때문입니다 (FR-128). 같은 프로젝트의 다른 멤버 기록은 빼야 합니다
수정 시각이 같으면 `project_id`가 큰 것을 먼저	58번 `list_projects`와 같은 규칙입니다. 두 화면의 순서가 같아집니다
세션은 오래된 것부터 정렬	화면이 시간 순서대로 막대를 쌓기 쉽습니다
UTC 그대로 보내고 요일은 화면이 나눔	서버가 사용자의 시간대를 몰라도 됩니다. 65번의 "오늘" 결정과 같은 방향입니다
프로젝트가 없으면 바로 `[]`	빈 `IN ()` 조건으로 쿼리를 보내지 않기 위해서입니다

6. 시험 (`tests/test_dashboard_service.py`: 5개를 추가해서 모두 12개)

시험	확인한 것
프로젝트가 없음	`[]`를 돌려줍니다
5개와 범위	내 것 7개, 삭제한 것, 남의 것, 멤버로 참여한 남의 것을 넣으면 `Mine 0, Shared, Mine 1, Mine 2, Mine 3`이 최근 수정순으로 5개 나옵니다
응답 칸	bpm `128.50`, 키 `A minor`가 들어 있고, 세션이 없으면 `[]`입니다
세션 범위와 순서	A에서는 내 1일 전, 3일 전 세션이 `[300, 100]`(오래된 것 먼저)으로 나오고, 8일 전 세션과 다른 사람의 세션은 빠집니다. B에서는 6.9일 전 세션이 포함됩니다
쿼리 수	프로젝트 5개와 세션 5개가 있어도 SELECT는 **2번**만 나갑니다

전체 시험 508개가 통과했고, `ruff` 검사도 통과했습니다.